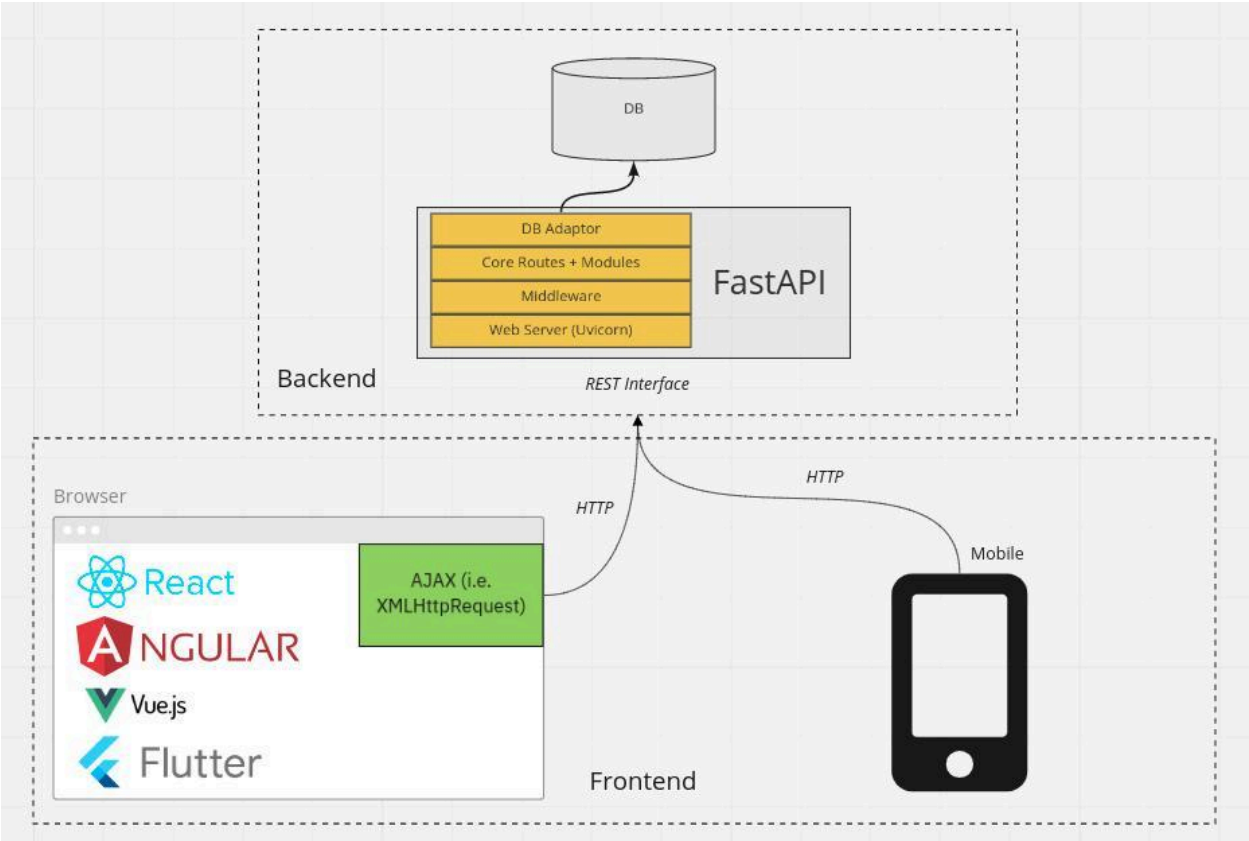
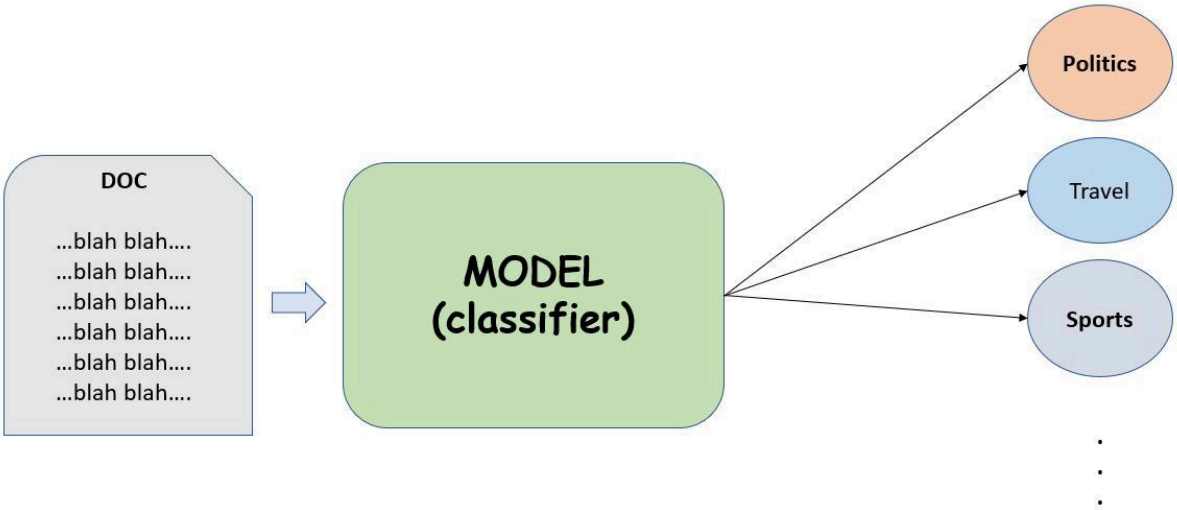
AI-powered system that reads a Jira issue and predicts its story points using ML + rules + LLM reasoning

Big Picture Architecture 👍

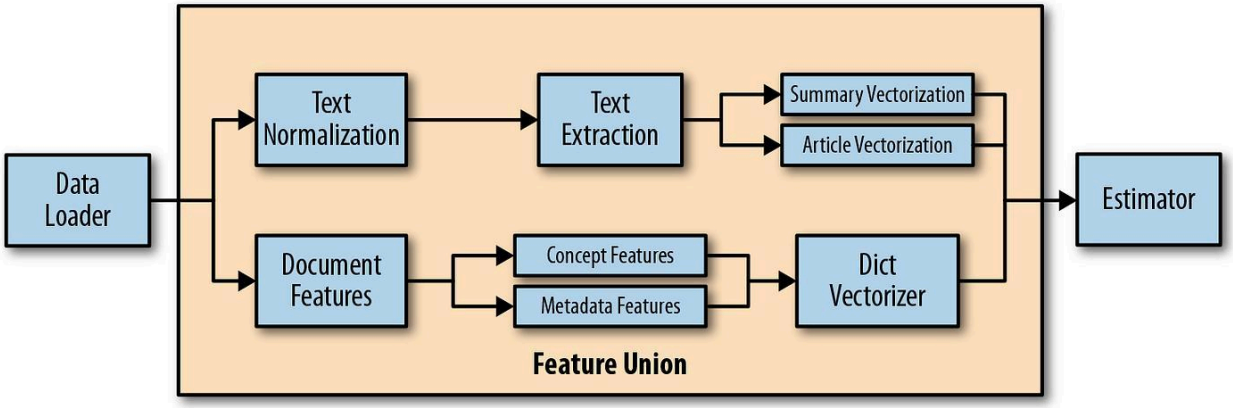


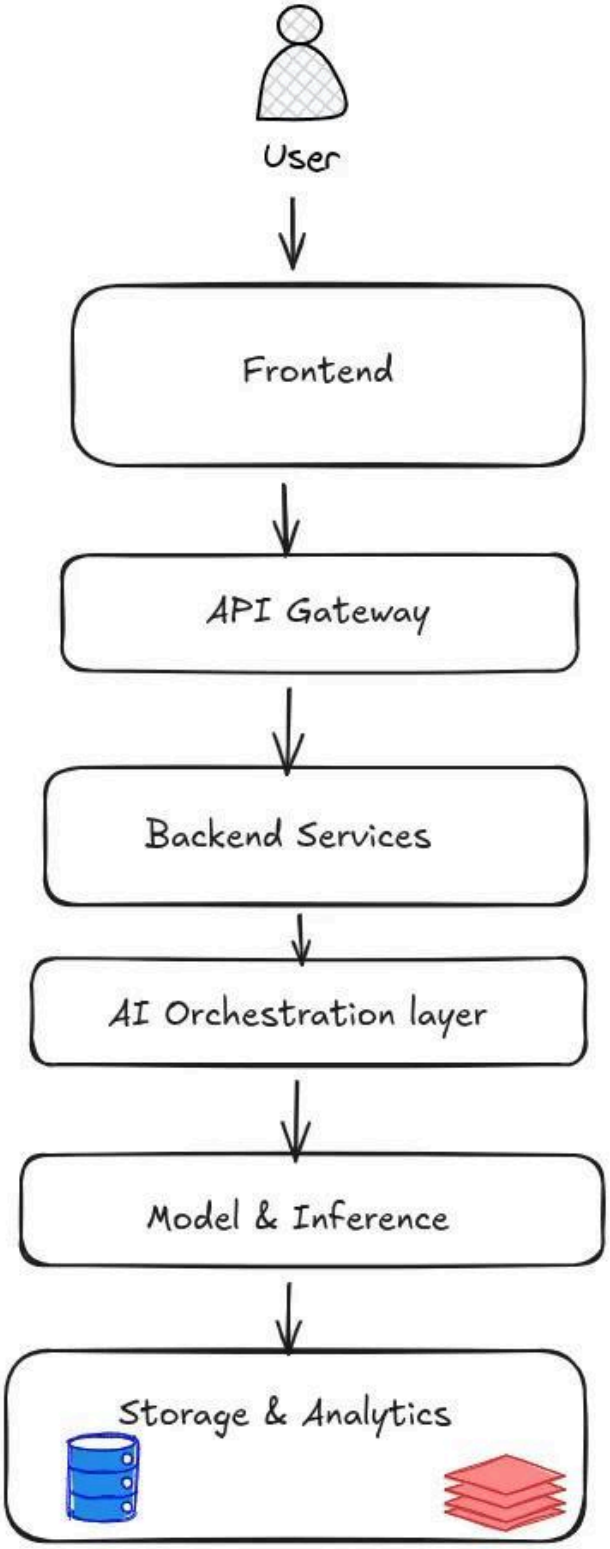
Jira
REST API REQUESTS
WEBHOOKS
Entity properties
UI tools
Issue fields
Your app
MODULES
Users


DOC
...blah blah....
...blah blah....
...blah blah....
...blah blah....
...blah blah....
...blah blah....
MODEL
(classifier)
Politics
Travel
Sports

User
(Resource owner)
Your client
(Consumer)
Jira
(Resource)
1
User performs action in client requiring Jira data
Request Jira data
Response with unauthorized token
Redirect to Jira
Present authentication request to user
User authorizes token
Redirect to client with access code
2
Request with access code
Response with access token
3
Request Jira data with access token
Response with Jira data
Present Jira data to user

Data Loader
Text Normalization
Text Vectorization
Feature Transformation
Estimator


Data Loader
Text Normalization
Text Extraction
Summary Vectorization
Article Vectorization
Document Features
Concept Features
Metadata Features
Dict Vectorizer
Feature Union
Estimator


User
Frontend
API Gateway
Backend Services
AI Orchestration layer
Model & Inference
Storage & Analytics

Full Flow (Step-by-Step Logic)

Let's follow what happens when you click **"Estimate"**

🟢 1 — User Input (Frontend)

👉 In your UI (`index.html`):

User enters:

SCRUM-1

👉 Clicks **Estimate**

🟡 2 — Frontend calls Backend

fetch(`/estimate/${issueKey}`)

👉 This sends request to:

/estimate/SCRUM-1

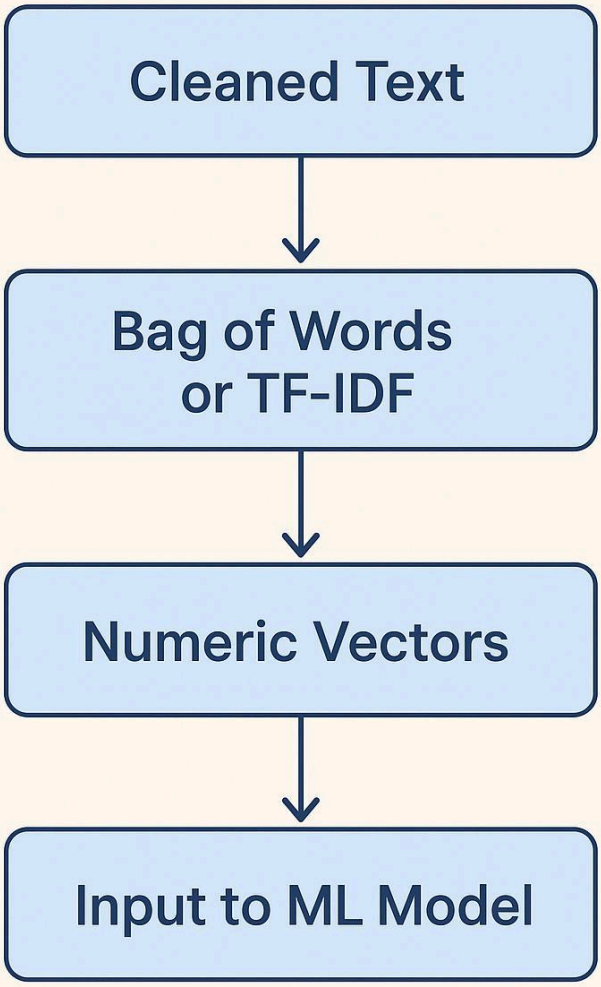
🔵 3 — FastAPI Backend (main.py)

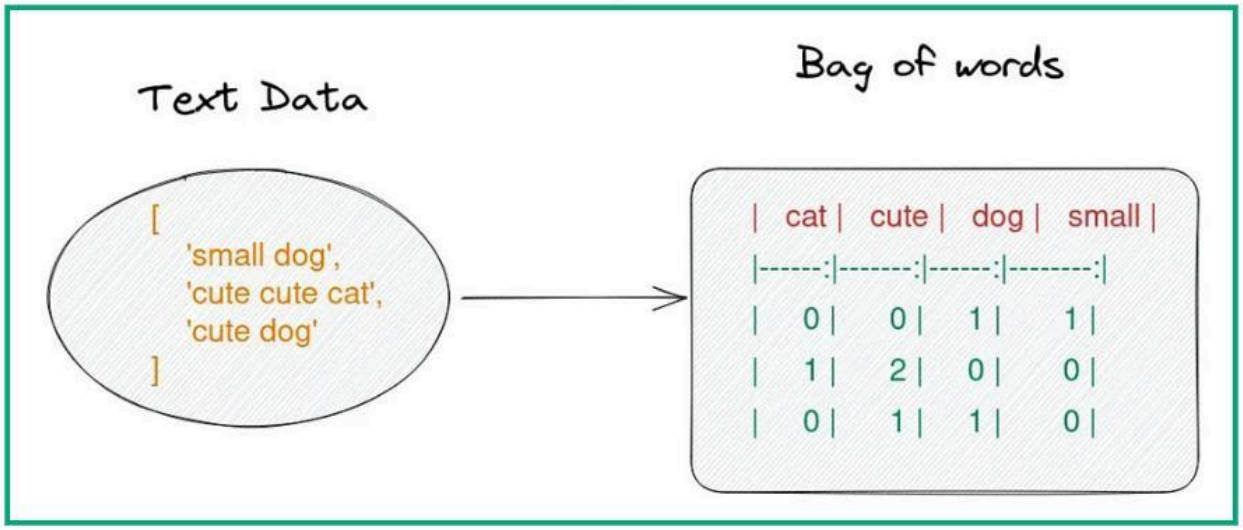
Your backend receives request:

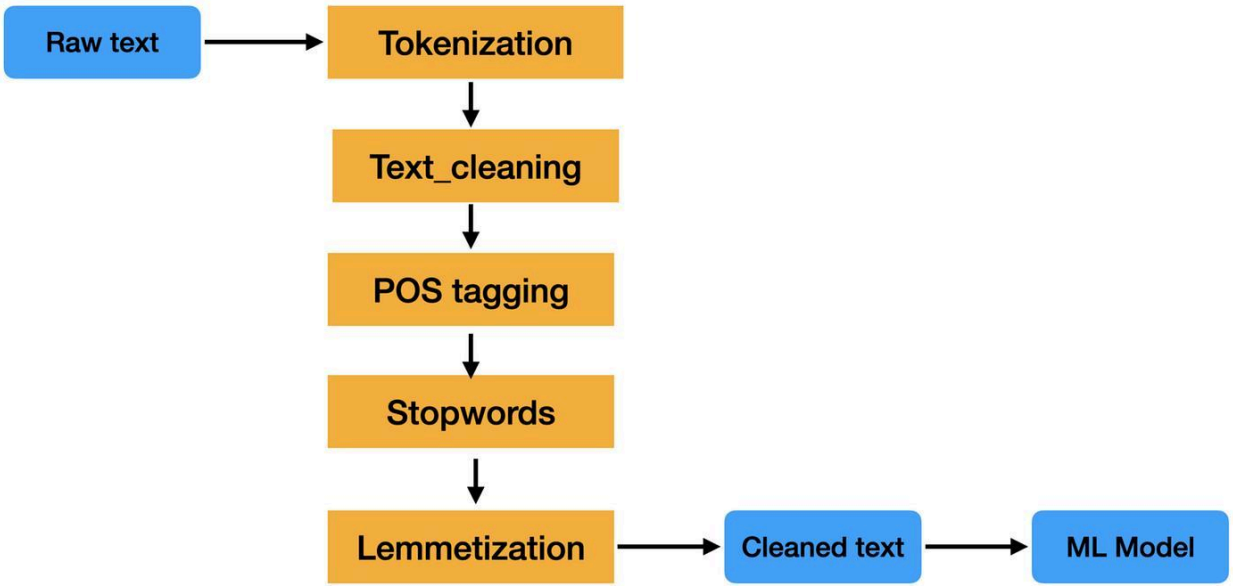
@app.get("/estimate/{issue_key}")

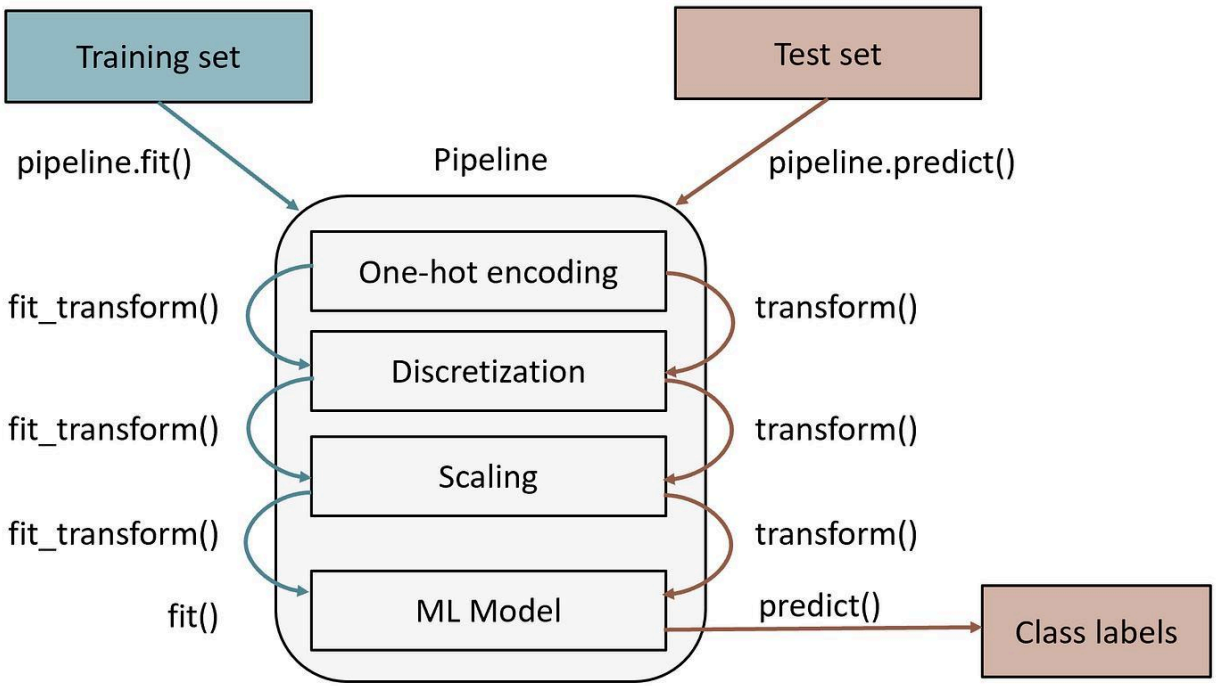
AI Prediction (ML Model) :

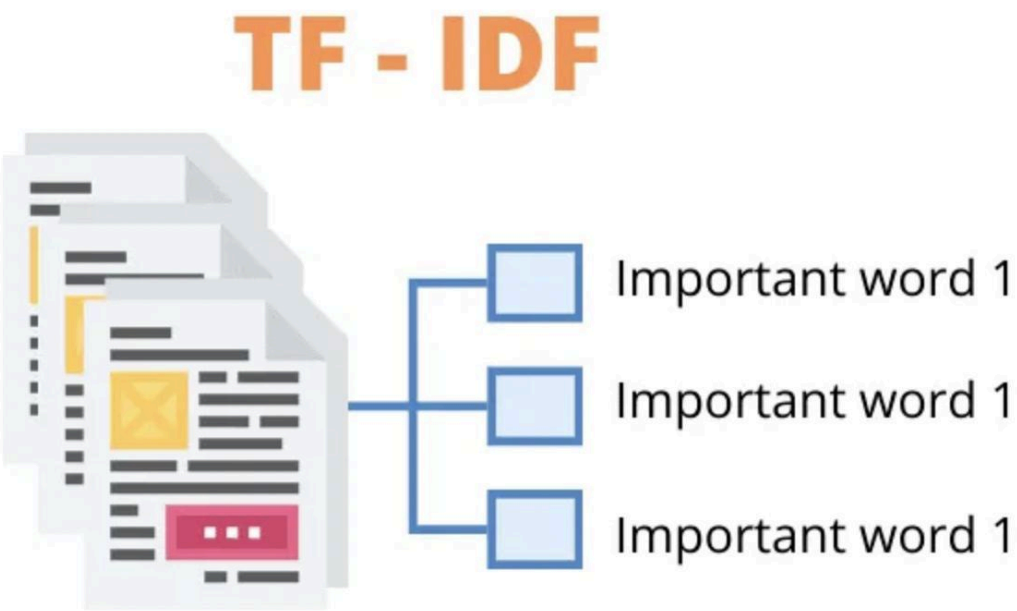
Text Vectorization in NLP for Beginners

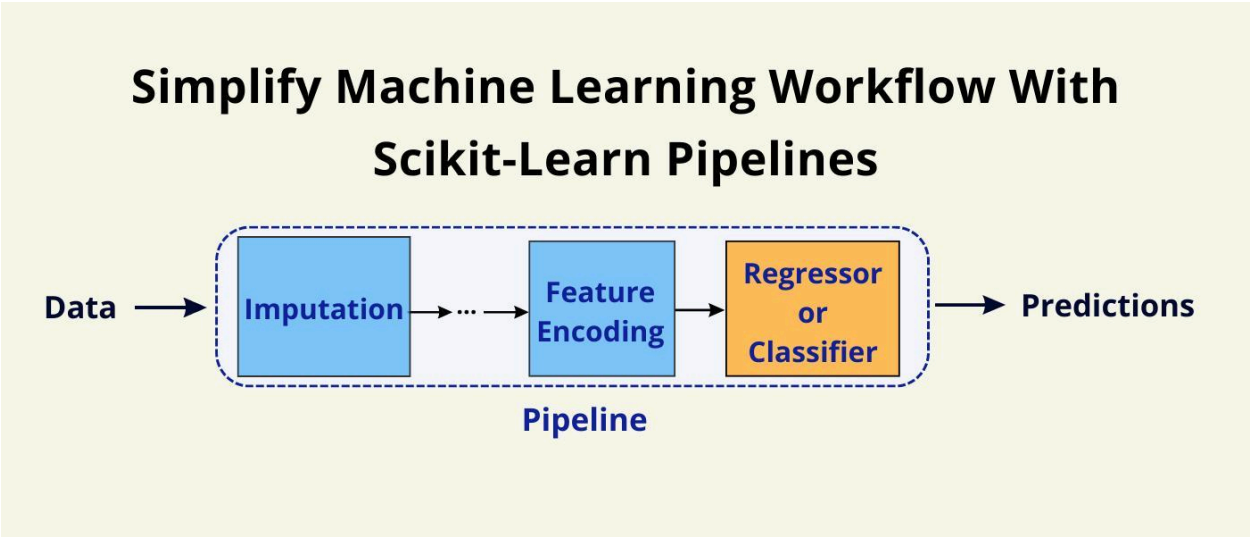



Text Data
Bag of words
[
'small dog',
'cute cute cat',
'cute dog'
]
cat	cute	dog	small
0	0	1	1
1	2	0	0
0	1	1	0


Raw text
Tokenization
Text_cleaning
POS tagging
Stopwords
Lemmetization
Cleaned text
ML Model


Training set
Test set
pipeline.fit()
Pipeline
pipeline.predict()
One-hot encoding
fit_transform()
transform()
Discretization
fit_transform()
transform()
Scaling
fit_transform()
transform()
fit()
ML Model
predict()
Class labels


TF - IDF
Important word 1
Important word 1
Important word 1



What happens:

```
X = vectorizer.transform([text])
prediction = model.predict(X)
```

Simple explanation:

👉 **Text → Numbers → Model → Prediction**

Example:

```
"Login feature with database"
→ converted to numbers
→ model predicts: 5 story points
```

Also calculates confidence:

```
confidence = max(probability) * 100
```

⚙️ Rule Engine (Extra Intelligence)

Your system doesn't depend only on AI.

It adds logic like:

- **Priority High → more points**
- **Security work → more effort**
- **Database work → more effort**

👉 Final result =

AI + Rules → Final Story Points

💾 Save Learning

save_estimation(issue_key, predicted, actual)

👉 Stores data for future learning

(Currently JSON → later MongoDB)

🔄 Update Jira

update_story_points(issue_key, predicted)

👉 Automatically updates Jira ticket

🤖 LLM Reasoning (Groq)

generate_llm_reasoning(summary, result)

👉 This explains:

> **"Why 5 story points?"**

Example:

Authentication + DB + Security → medium complexity

🟣 Send Response to UI

```
{
 "story_points": 5,
 "confidence": 23%,
 "ai_reasoning": "...",
 "llm_reasoning": "..."
}
```

🟢 UI Displays Result

You see:

- **Story Points ✅**
- **AI Prediction ✅**
- **Confidence ✅**
- **Explanation ✅**

🧩 Components of Project

---

1. 🎨 Frontend (index.html)

- Input field
- Button
- Displays result

---

2. ⚙️ Backend (FastAPI)

- API endpoints
- Business logic
- Integration layer

---

3. 🤖 AI Model

- Trained using scikit-learn
- Uses vectorizer + classifier

---

4. 🔗 Jira Integration

- Fetch issue
- Update story points

---

5. 🧠 LLM (Groq)

- Human-like explanation

---

6. 💾 Storage

- JSON file (current)
- MongoDB (next step)

🔥 Key Features of Project:

You combined:

✔ Machine Learning
✔ API Integration
✔ Backend Development
✔ Frontend UI
✔ Deployment
✔ LLM reasoning

👉 This is full-stack AI system

“This system takes a Jira issue, extracts its details, uses a trained ML model and rule-based logic to estimate story points, and then uses an LLM to explain the reasoning. It also updates Jira automatically and stores results for future learning.”

🏁 Final Understanding

👉 **Input → Jira → AI → Rules → LLM → Output**